

Licenciatura em Engenharia Informática e Multimédia

Infraestruturas Computacionais Distribuídas

Trabalho Prático 1 – Professor Cedric Grueau

24/04/2026

Jaime Rodrigues N°53123

Henrique Reguengo N°52935

Tiago Teixeira N°52864

Índice

Introdução	3
Diagrama Geral	
da Arquitetura	4
Descrição dos	
Nós da	
Arquitetura.....	5
Utilização dos	
Protocolos da	
Camada de	
Transporte	7
Diagrama de	
Classes	9
Conclusão	11
 Figura 1 -	
Diagrama da	
Arquitetura	4
Figura 2 -	
Diagrama	
de Classes	9
Figura 3 -	
Registrar	
Novo	
Jogador	12
Figura 4 - Log-in	12
Figura 5 -	
Gameplay	12

Introdução

O presente trabalho descreve a implementação de uma infraestrutura computacional distribuída que permite a realização de partidas remotas do jogo de estratégia **Pontos e Caixas**. O sistema foi desenhado para suportar a interação entre dois jogadores através de uma rede de computadores, utilizando uma arquitetura Cliente-Servidor baseada no protocolo TCP.

Os principais objetivos do sistema incluem:

- **Gestão de Identidade:** Registo e autenticação de utilizadores, permitindo o armazenamento de dados como alcunha, idade, nacionalidade e histórico de partidas.
- **Lógica de Jogo Centralizada:** O servidor atua como autoridade máxima, validando movimentos em grelhas retangulares (ex. 3 x 3 ou 5 x 5), detectando a conquista de caixas.
- **Persistência:** Utilização de XML e XSD para garantir a integridade dos jogos e das estatísticas de vitória/derrota.

Diagrama Geral da Arquitetura

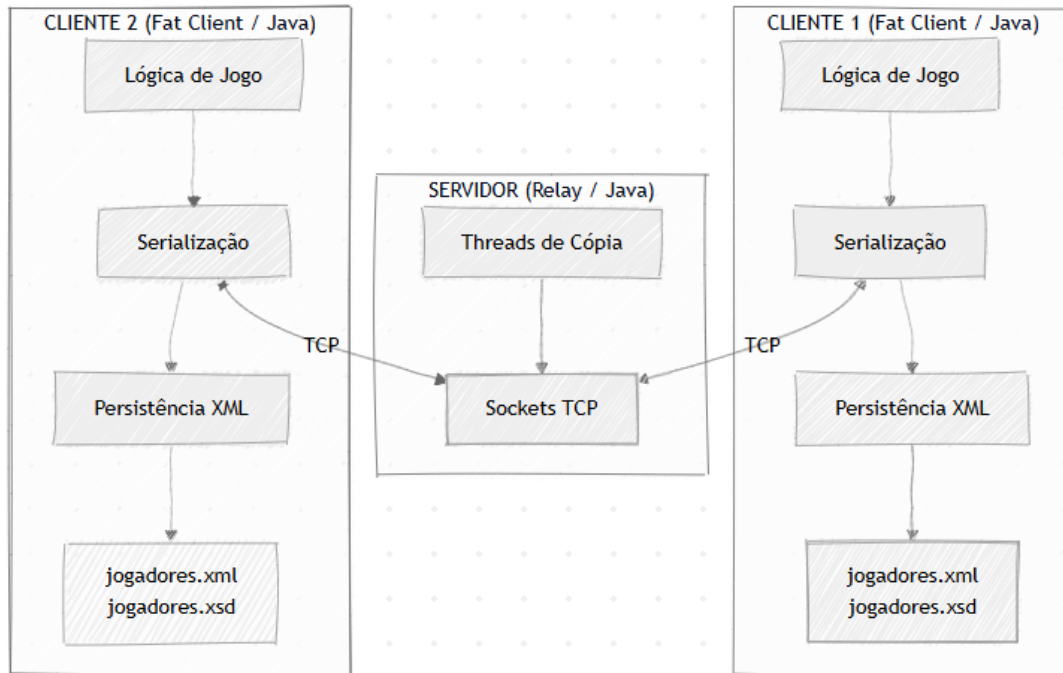


Figura 1 - Diagrama da Arquitetura

O sistema baseia-se numa arquitetura Cliente-Servidor de Retransmissão (Relay), onde a inteligência do jogo está totalmente descentralizada nos clientes (Fat Clients). Os principais componentes são:

- **Servidor de Retransmissão (Server):** Atua como um intermediário. A sua única função é estabelecer a ligação TCP inicial e reencaminhar os fluxos de bytes entre dois jogadores através de Threads. O servidor não processa a lógica de jogadas nem conhece o estado do tabuleiro.
- **Clientes (JogadorJogo):** São responsáveis pela execução da lógica de negócio. Cada cliente gere o seu próprio estado do objeto Jogo, valida as jogadas do utilizador e renderiza o tabuleiro na consola. A comunicação entre clientes é feita através de Serialização e Objetos, garantindo que

ambos os jogadores visualizem o mesmo estado do tabuleiro em tempo real.

- **Gestão de Turnos (Máquina de Estados):** A sincronização de turnos é controlada pelo próprio objeto Jogo. Ao encapsular a variável `vezAtual` dentro do objeto serializado, evita-se que ambos os clientes tentem jogar simultaneamente, criando uma fonte de verdade única para o estado da partida.
- **Camada de Persistência Local:** Ao contrário de sistemas centralizados, as estatísticas dos jogadores (vitórias, derrotas, tempo de jogo) são armazenadas localmente em cada cliente num ficheiro XML. A integridade destes dados é garantida por um esquema XSD, que define as regras estruturais e os tipos de dados permitidos.
- **Diferença entre XML e XSD:** O XML (Extensible Markup Language) funciona como uma base de dados local do sistema. É onde a informação é armazenada, permitindo que guarde e recorra a diferentes users diferentes. O XSD (XML Schema Definition) define a estrutura obrigatória, os tipos de dados permitidos e as restrições de cada campo, garantindo a integridade dos dados e impedindo que o sistema tente processar informação mal formatada ou corrompida. Em suma, o XML contém os dados, o XSD garante a validade desses mesmos dados.

Descrição dos Nós da Arquitetura

Nesta arquitetura cliente-servidor, as competências estão distribuídas de forma a garantir a integridade do estado do jogo e a persistência dos dados dos utilizadores.

Servidor (Concorrente e Relay):

O servidor foi implementado como um servidor concorrente, utilizando threads para gerir o emparelhamento de jogadores. Nesta solução, o servidor funciona como Relay Server (servidor de retransmissão).

Responsabilidades do Servidor:

- **Estabelecimento de Ligação:** Aceita conexões TCP no porto 5025 e atribui os números 1 e 2 referentes ao jogador 1 e jogador 2.
- **Encaminhamento de Objetos:** Atua como o canal de comunicação que recebe o objeto Jogo serializado de um cliente e retransmite instantaneamente para o oponente.
- **Gestão de Persistência (XML):** Utiliza a classe XMLReader para carregar dados, validar o login dos utilizadores e atualizar de forma persistente as estatísticas de vitórias, derrotas e tempo de jogo no ficheiro jogadores.xml.
- **Concorrência:** Gere o fluxo bidirecional de dados entre sockets sem bloquear a execução do sistema.

Cliente (Fat Client):

A solução utiliza um modelo de Fat Client. Ao contrário de um thin client, o executável do jogador possui uma cópia local da lógica do jogo (Jogo.java).

Responsabilidades do Cliente:

- **Processamento da Lógica:** O cliente é quem executa o método joga() e calcula localmente se uma aresta fechou um quadrado, atualizando a sua própria instância do tabuleiro e pontuação
- **Serialização de Estado:** Após cada jogada, o cliente serializa o objeto completo Jogo e envia-o ao servidor para que o oponente receba o estado atualizado.
- **Interface e Input:** Gere a interação com o utilizador via consola, desenhando o tabuleiro (printTabuleiro) e validando as coordenadas inseridas antes de as processar.
- **Controlo de Turnos Local:** Determina, com base no retorno da sua própria lógica, se o jogador tem direito a uma jogada extra (bónus) ou se deve aguardar a receção de um novo objeto via rede.

Utilização dos Protocolos da Camada de Transporte

Nesta solução, optou-se pela utilização do protocolo TCP (Transmission Control Protocol) no porto 5025.

- **Justificação do TCP:** O jogo Pontos e Caixas é um jogo baseado em turnos onde a integridade e a ordem dos dados são críticas. O TCP garante que o objeto Jogo, ao ser enviado de um cliente para o outro, chega sem erros, sem duplicação e na sequência correta. Uma perda de pacotes (comum em UDP) causaria uma dessincronização no estado do tabuleiro.
- **Porto Utilizado:** Foi selecionado o porto 5025, garantindo que não há conflito com serviços padrão do sistema operativo (portos well-known).

Formalização dos Protocolos ao Nível da Aplicação

Sintaxe e Estrutura das Mensagens

A comunicação é feita através da troca da classe Jogo, que implementa a interface Serializable.

- **Identificador de Versão:** `private static final long serialVersionUID = 1L;` (garante que ambos os nós usam a mesma definição da classe).
- **Conteúdo da Mensagem:** O objeto transporta a matriz `tabuleiro[][]` (arestas), a matriz `quadrados[][]` (donos das caixas) e as pontuações `pontos1` e `pontos2`.

Semântica (O significado das trocas)

1. **Handshake Inicial:** O servidor envia um char (“1” ou “2”) para definir a identidade de cada cliente.
2. **Sincronização de Estado:** Atualiza o estado atual do cliente A e envia a informação ao utilizador.
3. **Atualização de Estatísticas:** No final, o envio de dados para o XMLReader traduz o estado final do objeto em persistência no ficheiro jogadores.xml.

Temporização (Workflow)

O protocolo segue um modelo de Pedido/Resposta Alternado, mas com suporte a bónus:

- **Fluxo Normal:** Cliente A joga -> Envia Objeto -> Bloqueia em `in.readObject()` -> Cliente B recebe -> Joga -> Repete.
- **Regra de Bónus:** Se `objetoJogo.joga()` retornar true, o Cliente A não passa o turno, permitindo que ele envie múltiplas atualizações consecutivas antes de o cliente B recuperar o controlo de socket.

Diagrama de Classes

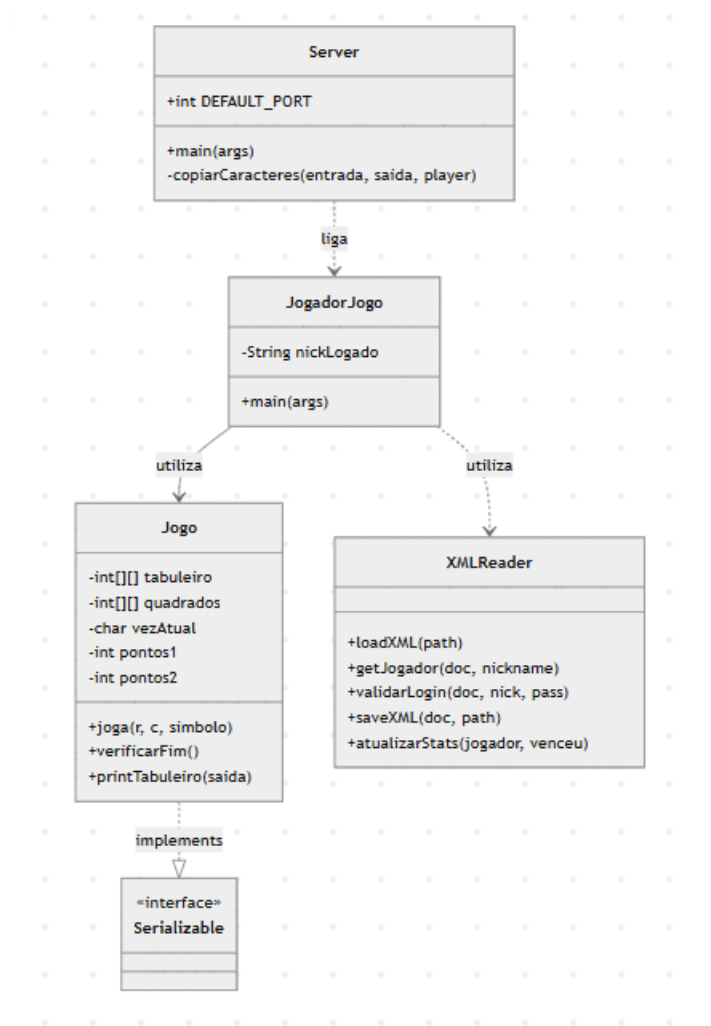


Figura 2 - Diagrama de Classes

O diagrama de classes ilustra a separação entre a lógica de negócio, a persistência de dados e a infraestrutura de comunicação. A arquitetura foi desenhada para ser um Fat Client, onde a inteligência do jogo reside no cliente, permitindo que o servidor permaneça agnóstico ao conteúdo das mensagens, ou seja, consegue ser utilizado em outros jogos sem uma única alteração.

O diagrama de Classes apresenta a estrutura da solução. Destaca-se a classe Jogo, que funciona como um DTO (Data Transfer Object) serializável, esta contém não só os dados do tabuleiro, mas também os métodos de validação de jogadas, garantindo que a lógica é idêntica em ambos os clientes. A persistência é gerida pela classe XMLReader, que assegura a conformidade dos dados locais com o esquema XSD definido. Esta organização permite uma elevada transparência de rede, onde a complexidade da comunicação TCP é isolada da experiência de jogo do utilizador.

Conclusão

A implementação deste sistema permitiu consolidar os conceitos de computação distribuída através do desenvolvimento de uma solução funcional do jogo “Pontos e Caixas”. A arquitetura adotada demonstrou ser eficaz para o objetivo proposto, apresentando as seguintes características relevantes:

- **Concorrência e Escalabilidade:** Através do uso de Threads no servidor de retransmissão, a solução é capaz de gerir múltiplos pares de jogadores em simultâneo sem interferência entre sessões, garantindo a escalabilidade vertical da aplicação.
- **Segurança e Integridade de Dados:** A utilização de esquemas XSD para validar os ficheiros XML locais garante que a persistência das estatísticas seja robusta, impedindo que o software processe dados corrompidos ou estruturalmente inválidos.
- **Expansibilidade:** O servidor foi desenhado como Relay Server, o que permite que a infraestrutura de rede possa ser reutilizada para outros jogos baseados em turnos com alterações mínimas no código do servidor.
- **Tolerância a Falhas:** Atualmente, a tolerância a falhas é limitada, uma vez que a interrupção da ligação TCP ou a queda do servidor central resulta na terminação imediata da partida.

Em suma, o projeto cumpre os requisitos de um sistema distribuído moderno, equilibrando a inteligência processual no cliente (Fat Client) com uma infraestrutura de comunicação fiável e persistente.

```
1. Login
2. Registo
2
Novo Nickname: yyy
Nova Password: bbb
Nacionalidade: Portugal
Idade: 20
🔌 Ligação estabelecida: Socket[addr=localhost/127.0.0.1,port=5025,localport=53293]
```

Figura 3 - Registrar
Novo Jogador

```
1. Login
2. Registo
1
Nickname: abc
Password: 12345
🔌 Ligação estabelecida: Socket[addr=localhost/127.0.0.1,port=5025,localport=53287]
```

Figura 4 - Log-in

```
0 . . . . .
1 | . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
🔌 Aguardando a jogada do oponente...

0 . . . . .
1 | | . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .

Sua vez! Digite a LINHA e a COLUNA da aresta:
2
1
|
0 . . . . .
1 | 2 | . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .

Fechaste um quadrado! Joga de novo.

Sua vez! Digite a LINHA e a COLUNA da aresta:
```

```
</jogador>

<jogador>
  <nickname>pot</nickname>
  <password>abc</password>
  <nacionalidade>Portugal</nacionalidade>
  <idade>20</idade>
  <foto>default.jpg</foto>
  <vitorias>0</vitorias>
  <derrotas>0</derrotas>
  <tempoTotal>0</tempoTotal>
</jogador>
</listaJogadores>
```

Figura 5 - Gameplay



Relatório do Trabalho Prático 1

Docente: Cedric Grueau

Jaime Rodrigues Nº53123

Henrique Reguengo Nº 52935

Tiago Teixeira Nº52864

Instituto Superior de Engenharia de Lisboa,
26 de abril de 2026